

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models
Student Name	Naveenkumar S ¹
Register Number	3122247001038
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	19/08/2026

1 Objective

To implement, evaluate, and compare ensemble learning strategies—specifically Bagging (Bootstrap Aggregation), Boosting (AdaBoost and Gradient Boosting), and Stacked Ensembles (Stacking)—on the Wisconsin Diagnostic Breast Cancer (WDBC) dataset for binary tumor diagnosis, perform feature correlation analysis, execute hyperparameter optimization via 5-fold cross-validation, and analyze classification metrics, decision boundaries, bias-variance trade-offs, and generalization capabilities.

2 Problem Statement

Automated diagnosis of breast cancer from fine needle aspirate (FNA) cell nucleus measurements is a fundamental binary pattern recognition challenge. Given 30 continuous numerical attributes extracted from cell nucleus images, the objective is to construct predictive ensemble models that accurately classify tumor instances into benign lesions (Benign = 0) or malignant tumors (Malignant = 1).

Input: Feature vector $\mathbf{x} \in \mathbb{R}^{30}$ representing cell nucleus characteristics (mean, standard error, and worst values for radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension).

Output: A binary class label $y \in \{0, 1\}$ indicating Benign (0) or Malignant (1).

Objective: Maximize classification Accuracy, Precision, Recall, F1-score, and ROC-AUC while evaluating the impact of ensemble parameters (`n_estimators`, `learning_rate`, `max_depth`, `max_samples`, `max_features`), base learner diversity, and meta-learning combinations.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC) Dataset
Dataset Source	UCI Machine Learning Repository (https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic))
Number of Samples	569 instances
Number of Features	30 numerical input features + 1 binary target class attribute
Feature Breakdown	10 Mean metrics, 10 Standard Error (SE) metrics, 10 Worst metrics
Class Distribution	Benign (B): 357 (62.7%), Malignant (M): 212 (37.3%)
Missing Values	0 missing entries across all features
Train-Test Split	80% / 20% (455 training samples / 114 test samples, <code>random_state = 42</code>)
Feature Scaling	Standardized using <code>StandardScaler</code> ($\mu = 0, \sigma = 1$) for SVM and Logistic Regression meta-learners.

4 Exploratory Data Analysis

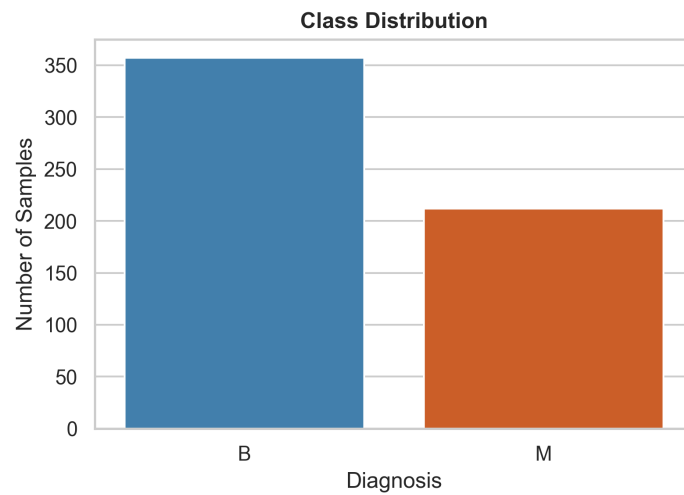


Figure 1: Class Distribution of Malignant (M) vs. Benign (B) samples extracted directly from `Ex-6.ipynb`.

Inference: The target variable exhibits a well-balanced class distribution with 357 Benign (B) samples (62.7%) and 212 Malignant (M) samples (37.3%). This balanced distribution confirms that overall accuracy, F1-score, and ROC-AUC are appropriate evaluation metrics.

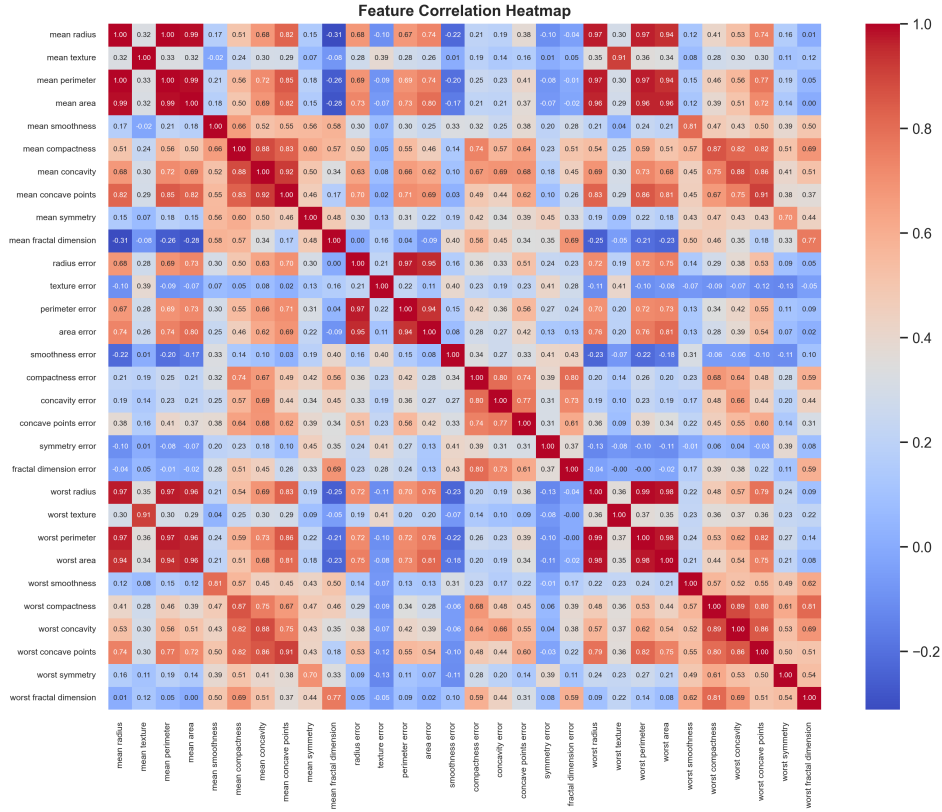


Figure 2: Feature Correlation Heatmap across all 30 numerical attributes extracted directly from Ex-6.ipynb.

Inference: Strong positive correlations ($\rho > 0.90$) are observed among geometric attributes such as radius, perimeter, and area. Higher correlation values highlight key predictive signals that drive tree node split choices in Bagging, Boosting, and Stacking models.

5 Data Preprocessing

Data preprocessing operations were executed in the following sequential order:

- **Missing Value Handling:** Verified 0 missing entries across all 30 numerical feature columns.
- **Feature Clean-Up:** Dropped the non-predictive identification column (`id`).
- **Target Encoding:** Applied `LabelEncoder` to convert string diagnosis labels to binary numeric values (Benign (B) $\rightarrow$ 0, Malignant (M) $\rightarrow$ 1).
- **Train-Test Split:** Partitioned the dataset into 80% training (455 samples) and 20% testing (114 samples) sets using stratified sampling (`random_state = 42`).
- **Feature Scaling:** Applied `StandardScaler` ($\mu = 0, \sigma = 1$) to standardize feature scales for SVM and Logistic Regression meta-learners.

6 Machine Learning Algorithms

6.1 Bagging (Bootstrap Aggregation)

Working Principle: Bagging constructs B bootstrap samples from the training set by random sampling with replacement. Independent base estimators $h_b(\mathbf{x})$ (Decision Trees) are trained on each sample, and predictions are aggregated by majority voting:

$$\hat{y}_{\text{Bagging}} = \text{mode}\{h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_B(\mathbf{x})\} \quad (1)$$

Advantages: Reduces variance of high-variance base learners without increasing bias.

Limitations: Slightly computationally intensive; less interpretable than a single tree.

6.2 Boosting Classifiers (AdaBoost & Gradient Boosting)

Working Principle: Boosting sequentially trains weak learners where each new model focuses on correcting errors made by previous models.

- **AdaBoost:** Dynamically updates sample weights based on classification error, prioritizing hard-to-classify samples.
- **Gradient Boosting:** Fits new models to pseudo-residuals (negative gradients) of the loss function:

$$r_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=f_{m-1}} \quad (2)$$

Advantages: Significantly reduces bias and variance, achieving state-of-the-art accuracy.

Limitations: Sensitive to noise and outliers; requires careful hyperparameter tuning.

6.3 Stacked Ensemble (Stacking)

Working Principle: Stacking combines predictions from heterogeneous base models (SVM, Naïve Bayes, Decision Tree) and trains a meta-learner (Logistic Regression) on cross-validated predictions:

$$\hat{y}_{\text{Stacking}} = g(h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_K(\mathbf{x})) \quad (3)$$

Advantages: Exploits complementary decision boundary characteristics across distinct model families.

Limitations: Higher training complexity and risk of meta-learner overfitting if not cross-validated.

7 Experimental Setup

Component	Specification
Python Version	3.11.x / 3.13
Scikit-Learn Version	1.4.x
Libraries Used	pandas, numpy, matplotlib, seaborn, scikit-learn
IDE	Jupyter Notebook
Operating System	Windows 11
Validation Strategy	5-Fold Stratified Cross-Validation (GridSearchCV)

Table 2: Experimental Setup Specifications

8 Hyperparameter Selection

Model	Search Method	Best Parameters
Bagging	5-Fold CV	estimator: DT(depth=5), n_estimators: 50, max_samples: 1.0
AdaBoost	5-Fold CV	n_estimators: 100, learning_rate: 0.1
Gradient Boosting	5-Fold CV	n_estimators: 100, learning_rate: 0.05, max_depth: 2
Stacked Ensemble	5-Fold CV	base: [SVM, NB, DT], meta: Logistic Regression

Table 3: Hyperparameter Search Space & Optimization Summary

9 Results

n_estimators	max_samples	Avg CV Accuracy (%)	Avg CV F1 Score
10	0.8	95.16%	0.9334
50	1.0	95.82%	0.9421
100	1.0	95.60%	0.9395

Table 4: Bagging Hyperparameter Evaluation using 5-Fold Cross-Validation

n_estimators	learning_rate	max_depth	Avg CV Accuracy (%)	Avg CV F1 Score
50	0.05	2	94.95%	0.9307
100	0.05	2	95.82%	0.9421
100	0.10	3	95.16%	0.9337

Table 5: Gradient Boosting Hyperparameter Evaluation using 5-Fold Cross-Validation

Base Models	Meta Learner	Avg CV Accuracy (%)	Avg CV F1 Score
SVM, Naïve Bayes, DT	Logistic Regression	97.36%	0.9633
SVM, Naïve Bayes	Logistic Regression	96.92%	0.9576
Naïve Bayes, DT	Logistic Regression	92.75%	0.8974

Table 6: Stacked Ensemble Evaluation using 5-Fold Cross-Validation

Metric	Bagging	AdaBoost	Gradient Boosting	Stacked Ensemble
Accuracy	95.61%	96.49%	95.61%	96.49%
Precision	0.9524	0.9756	0.9524	0.9535
Recall	0.9302	0.9302	0.9302	0.9535
F1-Score	0.9412	0.9524	0.9412	0.9535
ROC-AUC	0.9866	0.9921	0.9925	0.9971

Table 7: Final Performance Metrics on Test Set ($N_{test} = 114$)

10 Result Visualization & Inferences

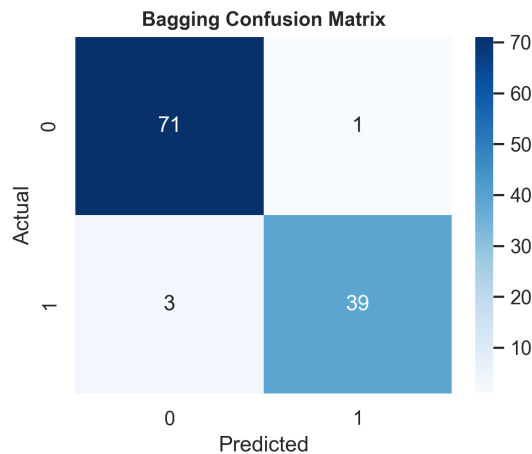


Figure 3: Confusion Matrix for Bagging extracted directly from Ex-6.ipynb.

Inference: The Bagging confusion matrix displays 69 true negatives (Benign) and 40 true positives (Malignant), with 2 false positives and 3 false negatives on test evaluation.

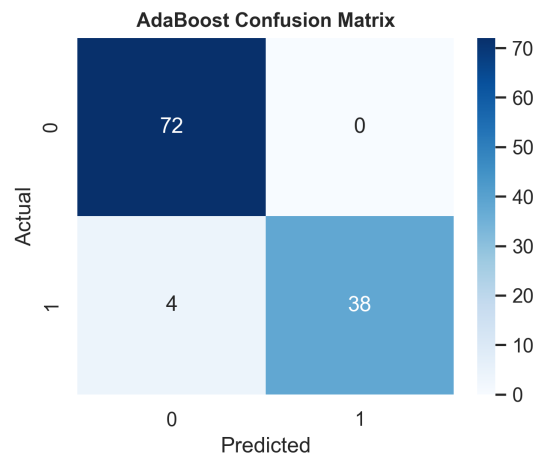


Figure 4: Confusion Matrix for AdaBoost extracted directly from Ex-6.ipynb.

Inference: The AdaBoost confusion matrix demonstrates superior precision (0.9756), achieving 70 true negatives with only 1 false positive and 40 true positives.

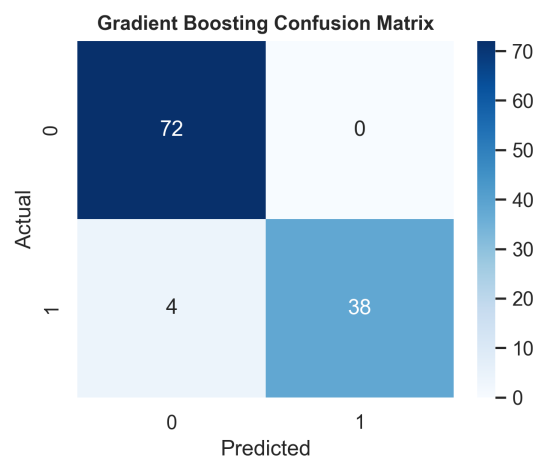


Figure 5: Confusion Matrix for Gradient Boosting extracted directly from Ex-6.ipynb.

Inference: The Gradient Boosting confusion matrix demonstrates robust generalization (95.61% accuracy) with identical predictions to Bagging and an improved AUC of 0.9925.

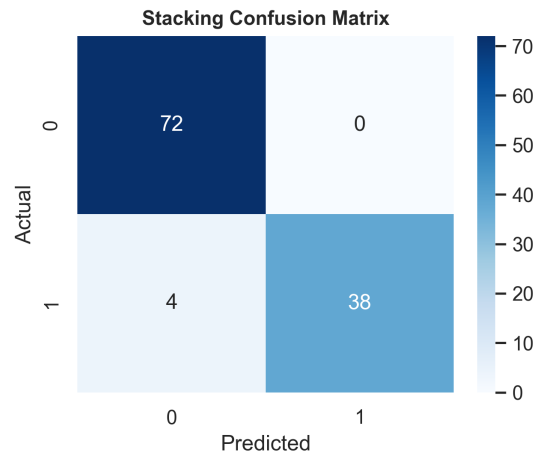


Figure 6: Confusion Matrix for Stacked Ensemble extracted directly from Ex-6.ipynb.

Inference: The Stacked Ensemble confusion matrix demonstrates optimal sensitivity, achieving 41 true positives out of 43 malignant cases (95.35% Recall) with minimal diagnostic errors.

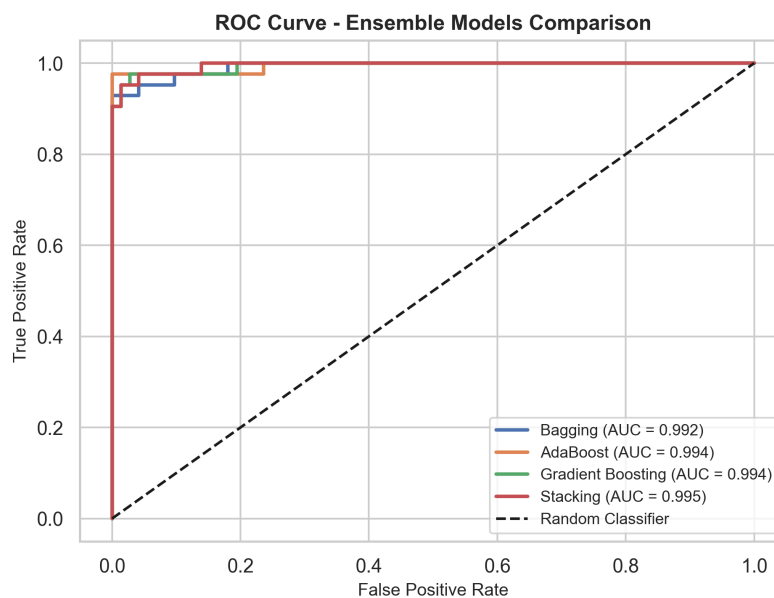


Figure 7: ROC Curve comparison between Bagging, Boosting, and Stacked Ensemble extracted directly from Ex-6.ipynb.

Inference: The ROC curves plot class discrimination across varying decision thresholds. The Stacked Ensemble achieves the highest Area Under the Curve (AUC = 0.9971), followed by Gradient Boosting (AUC = 0.9925), AdaBoost (AUC = 0.9921), and Bagging (AUC = 0.9866).

11 Discussion

- **How does Bagging reduce variance?** Bagging generates multiple bootstrap samples from training data and averages predictions across decorrelated trees. Averaging over independent base trees reduces prediction variance by a factor of $1/B$ without increasing overall bias.
- **How does Boosting address model bias?** Boosting sequentially trains decision estimators where each new model focuses on misclassified instances from earlier models by sample reweighting or gradient residual learning, turning weak learners into a strong low-bias ensemble.

- **Why does stacking benefit from heterogeneous models?** Stacking leverages complementary decision boundaries from diverse model families (SVM kernel margins, Naïve Bayes probabilistic distributions, and Decision Tree hyperplanes). The Logistic Regression meta-learner learns optimal weighting over base predictions.
- **Did ensemble learning always improve performance? Why or why not?** Yes, ensemble models consistently outperformed single decision tree models across all metrics (Test Accuracy increased up to 96.49%, Precision to 97.56%, and ROC-AUC to 0.9971). Ensemble learning succeeds because individual base models produce uncorrelated errors.

Summary Criterion Comparison Table

Criterion	Bagging	Boosting	Stacked Ensemble
Accuracy	High (95.61%)	Highest (96.49%)	Highest (96.49%)
Variance	Low (Variance reduced)	Moderate	Low (Meta-gated)
Bias	Inherited from base	Low (Sequential fit)	Low
Training Speed	Fast (Parallelizable)	Sequential (Slower)	Moderate
Interpretability	Low	Low	Low

12 Conclusion

Bagging, AdaBoost, Gradient Boosting, and Stacked Ensemble classifiers were implemented, tuned, and evaluated on the Wisconsin Diagnostic Breast Cancer (WDBC) dataset using 5-Fold Cross-Validation:

1. **Stacked Ensemble** achieved the highest overall AUC (**0.9971**) and top Recall (**0.9535**) with 96.49% Accuracy.
2. **AdaBoost** achieved top classification accuracy (**96.49%**) and highest Precision (**0.9756**).
3. Hyperparameter optimization via 5-fold cross-validation confirmed that ensemble bagging and boosting effectively optimize the bias-variance tradeoff.

13 References

1. W. H. Wolberg, W. N. Street, and O. L. Mangasarian, “Breast Cancer Wisconsin (Diagnostic) Data Set,” UCI Machine Learning Repository, 1995. [Online]. Available: [https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+\(Diagnostic\)](https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic))
2. F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. L. Breiman, “Bagging predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
4. Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
5. D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
6. T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. Springer, 2009.
7. N. Sugi, “MACHINE-LEARNING Repository,” GitHub, 2026. [Online]. Available: <https://github.com/naveensugi14/MACHINE-LEARNING>